

INDIAN INSTITUTE OF TECHNOLOGY (INDIAN SCHOOL OF MINES), DHANBAD

Winter End-Semester Examination, Session 2024-25

Examination: II UG (Section: A, B, C, D)

Subject: Computer Programming (NCSV101)

Time: 2 Hours

Max. Marks: 50

Instructions:

- (i) Answer ALL Questions.
- (ii) Attempt all sub-parts of a same question at same place and in order.
- (iii) Assume suitable data wherever not provided.

Q. No.	Question	Mark
1. (a)	Assume that there is a file "input.txt" with n integers where the value of n is already given. Write a C program to read the file and store all n integers in an array. Then write all odd integers into a file "odd.txt" and all even integers into a file "even.txt".	2+4
(b)	Consider the following definition of a union: <pre> union car{ char model[15]; float price_in_lakhs; struct property { char colour[8]; int no_of_seat; int max_speed; } p; } c1; </pre> What is the size of the union variable c1, assume that size of a char is 1 byte, size of an int is 4 bytes, and size of float is 4 bytes.	2
(c)	Define a structure to store a point on 2D plane (represented with x and y value). Write a C program to take n such points on 2D plane from user where n is also a user input. Find the top-most and left-most points. Assume that all points are distinct.	5
2. (a)	(i) Differentiate between arrays, pointers, and structures based on how can each be initialized? (ii) What are the valid forms of nesting in structures and functions explain with examples?	3+3
(b)	You are given the following structure definition: <pre> struct Student { char name[50]; int age; float marks; }; </pre> (i) Write a function displayStudent that accepts a Student structure by value and prints the student's details. (ii) Write another function updateMarks that accepts a pointer to a Student structure and updates the student's marks by adding 5 bonus points. (iii) In the main() function, demonstrate calling both displayStudent and updateMarks appropriately.	2+3 +1

Q. No.	Question	Mark
3. (a)	If a and b have been declared as integers and p and q as pointers to integers, then state errors with proper explanations, if any, in the given statements: (i) *p = &b; (ii) q = &*a; (iii) a = q-p; and (iv) p = &q;. If no such error presents, then write the proper reason for this.	4
(b)	Define a structure that describes a point in a two-dimensional Cartesian plane. Write a C function which takes addresses of two points as its arguments and returns the midpoint of the line segment joining these two points.	4
(c)	Write a C program that reads an existing file (named "DATA") containing only integers and appends at its end the average of all the integers. Use appropriate library function(s) to handle error situations which may arise during file I/O operations.	5
4. (a)	Consider the following recursive C function with appropriate header files: <pre>void analyzeRecursion(int n) { if (n <= 0) { printf("Base: %d\n", n); return; } printf("Pre: %d\n", n); analyzeRecursion(n - 2); printf("Post: %d\n", n); analyzeRecursion(n - 3); } int main() { analyzeRecursion(5); }</pre> List the sequence of "Pre: ...", "Post: ..." and "Base: ..." outputs that would be printed to the console/terminal.	4
(b)	You are given two integer arrays, potentially of different lengths, for example: <pre>int dataSet1[] = {10, 20, 30, 40}; int dataSet2[] = {5, 15, 25, 35, 45, 55};</pre> Write a C code to perform the following: (i) Determine the length of the shorter of the two arrays (dataSet1 and dataSet2). Dynamically create a third integer array (let's call it elementSumArr) that is exactly large enough to hold this number of elements (i.e., the length of the shorter array). (ii) Using only pointer arithmetic (i.e., don't use array bracket notation like arr[i]), populate the elementSumArr. Each element in elementSumArr should be the sum of the corresponding elements from dataSet1 and dataSet2. You should only process elements up to the length of the shorter array. (For the example above, the shorter length is 4, so elementSumArr should contain {10+5, 20+15, 30+25, 40+35} which is {15, 35, 55, 75}). (iii) Print the contents of elementSumArr to verify the element-wise sums.	2+2 +1
(c)	Analyse the code snippet below and examine the configureSensor function. What is the main problem or potential issue in the following code? Explain why this is considered incorrect in C. <pre>int* configureSensor() { int sensorId = 42; printf("Function scope: sensorId (%d)\n", sensorId); return &sensorId; } int main() { int *sensorPtr; printf("Configuring sensor...\n"); sensorPtr = configureSensor(); printf("Configuration done. Pointer received: %d\n", *sensorPtr); }</pre>	3